Software Requirements Specification

for

Smart Subscription Manager

Prepared By

Sahil Sharma (GEU Roll No. 44)
Samaksh (GEU Roll No. 45)
Tanishk Jain (GEU Roll No. 61)

Submitted To

Mr. Jyotir Moy Chatterjee

Department of Computer Science & Engineering
Graphic Era (Deemed to be University)

February 2026

Revision History

Name	Date	Reason For Changes	Version
Sahil Sharma, Samaksh, Tanishk Jain	January 2026	First draft written	0.1
Sahil Sharma, Samaksh, Tanishk Jain	February 2026	Reviewed and finalised for submission	1.0

1. Introduction

1.1 Purpose

This document describes the software requirements for the Smart Subscription Manager, a full-stack web application designed to help users track, manage, and monitor all their digital subscriptions in one centralized platform. It covers what the system is supposed to do, how it should behave, and what users can expect from it.

We are building this as a mini project for our 3rd year B.Tech in Computer Science Engineering. This document is meant to act as a reference for development, testing, and evaluation by our faculty supervisor.

1.2 Document Conventions

We have followed the IEEE 830 SRS format for writing this document. The conventions used are:

- Each functional requirement is given an ID like REQ-F-01, REQ-F-02, and so on.
- Each non-functional requirement (like performance, security) is given an ID like REQ-NF-01.
- If something has not been decided yet, we have written TBD (To Be Determined) in that place.
- Bold text is used to highlight important terms or requirement IDs.
- Higher-level requirements include all their sub-requirements unless stated otherwise.

1.3 Intended Audience and Reading Suggestions

This document is written for the following readers:

- Our development team (Sahil, Samaksh, Tanishk): We should read everything, especially Sections 3, 4, and 5 to understand exactly what we need to build.
- Faculty supervisor (Mr. Jyotir Moy Chatterjee): Sections 1, 2, and 4 give a good overview of the project scope and the features being implemented.
- Anyone testing the project: Sections 4 and 5 have the detailed requirements that test cases should be based on.

We suggest reading Section 1 and Section 2 first to understand what the project is about, then moving to Section 4 for the feature details.

1.4 Product Scope

Smart Subscription Manager is a full-stack web application that brings together several subscription management features in one place. The main things it does are:

- Allows users to register, log in securely, and manage their account.
- Enables users to add, edit, and delete subscriptions (e.g., Netflix, Spotify, AWS).
- Tracks renewal dates and sends automated renewal reminder alerts before a subscription expires.
- Provides a dashboard showing total subscriptions, total monthly cost, and upcoming renewals.
- Calculates financial exposure — total expected spending for the next 30 days across all subscriptions.
- Identifies potentially unused subscriptions based on logged usage data.
- Supports categorization of subscriptions (Entertainment, Cloud, Utilities, etc.) with risk levels.

Most existing tools like app store subscription pages only show a single service's billing. Smart Subscription Manager consolidates all subscriptions across all platforms into one easy-to-use interface with automated alerts and analytics. This project covers topics from Software Engineering, Web Development, Database Systems, and Backend API design.

1.5 References

- E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," Communications of the ACM, vol. 13, no. 6, 1970.
- JSON Web Tokens (JWT) RFC 7519. Available: https://datatracker.ietf.org/doc/html/rfc7519

- Node-cron Documentation. Available: https://github.com/node-cron/node-cron
- Express.js Official Documentation. Available: https://expressjs.com/
- React Official Documentation. Available: https://react.dev/
- PostgreSQL Official Documentation. Available: https://www.postgresql.org/docs/
- IEEE Std 830-1998, "IEEE Recommended Practice for Software Requirements Specifications."

2. Overall Description

2.1 Product Perspective

Smart Subscription Manager is a brand new project. It is not based on or replacing any existing system. The idea originated from the growing problem of subscription fatigue — users subscribe to many services and lose track of renewal dates, leading to unexpected charges.

Unlike a single-page browser app, this project runs on a 3-tier architecture. It has a React frontend, a Node.js + Express backend, and a PostgreSQL database. These three layers communicate through a REST API. Internally, the app has these main parts:

- Auth Module: Handles user registration, login, and JWT-based session management.
- Subscription Module: Full CRUD operations for managing subscriptions.
- Alert & Scheduler Module: A node-cron background job that checks renewal dates daily and creates alerts.
- Notification Module: Stores and delivers notifications to the user based on triggered alerts.
- Dashboard Module: Aggregates subscription data and displays analytics to the user.
- Usage Log Module: Records subscription usage events to identify potentially unused subscriptions.

2.2 Product Functions

Here is a summary of what Smart Subscription Manager can do:

- Allow users to register and log in securely using email and password (hashed with bcrypt, sessions managed via JWT).
- Let the user add subscriptions by selecting a service, plan, category, billing cycle, renewal date, cost, and payment method.
- Display a dashboard with total subscriptions, monthly cost, financial exposure for the next 30 days, and upcoming renewals within 7 days.
- Automatically run a daily background job (node-cron) that checks for subscriptions renewing within 3 days and creates alerts.
- Show renewal reminder alerts and notifications in the user's dashboard.
- Allow the user to edit or delete any of their subscriptions.
- Log subscription usage events so that unused subscriptions can be identified.
- Categorize subscriptions and display them with category-level risk information.

2.3 User Classes and Characteristics

We expect two main types of users for this project:

Registered End User (Main target user):

Someone who manages multiple digital subscriptions (streaming, cloud, utilities) and wants to track their spending and avoid surprise renewals. They interact with the dashboard, add/edit subscriptions, and receive alerts. The interface should be simple enough that no instructions are needed.

Faculty / Academic Evaluator:

Our professor who will evaluate the project. They will check the correctness of the architecture, database design, API implementation, and that the automated alerting logic works correctly. They may review both the code and this SRS document.

2.4 Operating Environment

Item	Details
Frontend Hosting	Vercel
Backend Hosting	Render
Database Hosting	Railway or Supabase (PostgreSQL)
Supported Browsers	Google Chrome 110+, Firefox 110+, Microsoft Edge 110+, Safari 16+
Supported Devices	Windows, macOS, Linux desktop/laptop; Android and iOS mobile
Internet Required?	Yes — for all API calls between frontend and backend
Frontend Framework	React.js with React Router
Backend Framework	Node.js with Express.js
Database	PostgreSQL (or MySQL)
Scheduler	node-cron
Authentication	JSON Web Tokens (JWT)
CSS Framework	Tailwind CSS
HTTP Client	Axios
Build Tool	Vite (frontend)

2.5 Design and Implementation Constraints

These are the limitations and restrictions we had to keep in mind while building the project:

- Technology stack: We decided on React + Node.js + Express + PostgreSQL from the beginning, so all code must follow this stack.
- JWT-based authentication: No sessions or cookies are used. Every protected API request must carry a valid JWT in the Authorization header.
- Passwords must be hashed: Raw passwords must never be stored. bcrypt is used with a salt factor of 10.
- SQL injection prevention: All database queries must use parameterized queries. Raw string interpolation in SQL is not allowed.
- Scheduler runs server-side: The node-cron job runs on the backend. The frontend does not perform any scheduling.
- Database schema must use foreign keys: All relationships defined in the ER diagram must be enforced at the database level.
- No paid APIs or external services: All components use free-tier plans or open-source libraries.
- Version control: All code must be committed to a GitHub repository using Git.

2.6 User Documentation

The following documentation will be available with the project:

- README.md on GitHub: Explains how to install the project, run it locally, and configure environment variables. Available at the project's GitHub repository.
- On-screen guidance: The website includes labels, placeholder text, and validation messages to guide users without a separate manual.
- This SRS document: Submitted to our faculty as the formal technical documentation for the project.

2.7 Assumptions and Dependencies

Assumptions:

- We assume users have a working internet connection while using the app.
- We assume the free tiers of Vercel, Render, and Railway/Supabase will be available during demos and testing.
- We assume users will use a modern browser that supports ES2020+ JavaScript.
- We assume the JWT secret key is stored securely as an environment variable and is never pushed to GitHub.

Dependencies:

- React.js: The entire frontend depends on this library.
- Express.js: All REST API endpoints are built on this framework.
- PostgreSQL: All persistent data is stored here. A schema change would require migration scripts.
- node-cron: The automated renewal reminder system depends entirely on this scheduler.
- bcrypt: Password hashing is handled by this library. A change in bcrypt would require re-hashing all passwords.
- JWT: All authentication flows depend on this token standard.

3. External Interface Requirements

3.1 User Interfaces

Smart Subscription Manager is a Single-Page Application (SPA). Here is how the user interacts with it:

- Navbar (Navbar.jsx): Present on all pages after login. Contains links to Dashboard, My Subscriptions, Add Subscription, Alerts, and Logout.
- Login / Register Pages (Login.jsx, Register.jsx): Simple forms for user authentication. Validation errors are shown below the relevant field.
- Dashboard (Dashboard.jsx): Shows summary statistics cards (total subscriptions, monthly cost, upcoming renewals, financial exposure) and a list of upcoming renewals. Data is fetched from the backend on load using useEffect.
- Subscription List Page: Shows all user subscriptions in a table with edit and delete actions per row.
- Add/Edit Subscription Page (AddSubscription.jsx, EditSubscription.jsx): Forms with dropdowns for service, category, plan, billing cycle, and date pickers for renewal date.
- Alerts Page: Shows all pending renewal alerts sorted by date, with status indicators.
- Responsive layout: The app works on desktop and mobile screens (minimum width 375px).
- Error messages: If something goes wrong (e.g., login fails, API error), a clear message is shown on screen.

3.2 Hardware Interfaces

Smart Subscription Manager does not connect directly to any hardware. But it relies on the following:

- Internet connection: Required for all communication between the frontend and backend.
- Screen: Minimum resolution of 360x640 pixels. The app looks best on 1366x768 or larger screens.
- Keyboard and mouse (or touchscreen): Standard input is all that is required. No special hardware is needed.

3.3 Software Interfaces

Smart Subscription Manager communicates with the following software systems:

PostgreSQL Database:

- Type: Relational database
- Connection: Via the pg (node-postgres) library from the backend
- What it does: Stores all persistent data including users, subscriptions, categories, services, plans, payments, alerts, notifications, and usage logs.

node-cron Scheduler:

- Type: Background job scheduler (runs inside Node.js process)
- What it does: Runs a daily job at midnight (cron expression: 0 0 * * *) that queries the database for subscriptions renewing within 3 days and inserts corresponding alert records.

Axios HTTP Client:

- Type: JavaScript HTTP client library
- What it does: Used on the frontend to make all REST API calls to the backend. JWT is attached in the Authorization header for protected routes.

3.4 Communications Interfaces

- All API requests between the React frontend and the Node.js backend use HTTPS in production.
- The frontend uses Axios to make HTTP GET, POST, PUT, and DELETE requests to the REST API.
- JWT tokens are passed in the HTTP Authorization header as a Bearer token on all protected endpoints.
- The backend uses the pg library to communicate with PostgreSQL over a connection pool.
- The database accepts connections only from the backend server, not from the public internet.
- CORS is configured on the backend to allow requests only from the deployed frontend URL.

4. System Features

4.1 User Authentication

4.1.1 Description and Priority

This module handles user registration and login. Users must create an account and log in before they can access any other feature. Passwords are stored as bcrypt hashes and sessions are managed with JWT tokens. Priority: High.

4.1.2 Stimulus / Response Sequences

- User visits the registration page -> Fills in name, email, phone, password, preferred currency, and timezone -> Clicks Register -> Backend hashes password and stores user -> JWT returned -> User redirected to dashboard.
- User visits the login page -> Enters email and password -> Backend verifies password against hash -> JWT returned on success -> User redirected to dashboard.
- User tries to access a protected page without a valid JWT -> Backend returns 401 Unauthorized -> Frontend redirects to login.
- User clicks Logout -> JWT is removed from frontend storage -> User redirected to login page.

4.1.3 Functional Requirements

Req. ID	Requirement
REQ-F-01	The app must provide a registration form collecting: name, email (unique), password, phone, preferred currency, and timezone.
REQ-F-02	Passwords must be hashed using bcrypt with a minimum salt factor of 10 before being stored in the database. Plain text passwords must never be stored.
REQ-F-03	On successful login, the backend must generate and return a signed JWT containing the user's ID and an expiration time.
REQ-F-04	All protected API endpoints must validate the JWT from the Authorization header before processing the request.
REQ-F-05	If the JWT is missing, invalid, or expired, the API must return HTTP 401 and the frontend must redirect the user to the login page.
REQ-F-06	The registration form must validate that the email is in the correct format and that the password meets a minimum length of 8 characters before submission.

4.2 Subscription Management

4.2.1 Description and Priority

This is the core feature of the app. It allows authenticated users to add, view, edit, and delete their subscriptions. Each subscription is linked to a service, a plan, a category, and a payment method. Priority: High.

4.2.2 Stimulus / Response Sequences

- User goes to Add Subscription page -> Selects service, plan, category, billing cycle, payment method, sets renewal date and price -> Clicks Save -> Subscription stored in database -> User redirected to subscription list.
- User views the subscriptions list -> All their subscriptions are shown with service name, price, renewal date, billing cycle, and status.
- User clicks Edit on a subscription -> Pre-filled form is shown -> User updates fields -> Clicks Update -> Database record updated.
- User clicks Delete on a subscription -> Confirmation prompt shown -> User confirms -> Subscription removed from database.

4.2.3 Functional Requirements

Req. ID	Requirement
REQ-F-07	Authenticated users must be able to add a new subscription with the following fields: service, subscription plan, category, payment method, billing cycle, start date, renewal date, price, currency, status (active/paused/cancelled), auto-renew flag, importance level, and optional notes.
REQ-F-08	The subscriptions list must show only the subscriptions belonging to the currently logged-in user. No user should be able to see or access another user's subscriptions.
REQ-F-09	Users must be able to edit any field of an existing subscription they own.
REQ-F-10	Users must be able to delete any subscription they own. Deletion must be permanent and remove the record from the database.
REQ-F-11	Each subscription must be associated with a category (e.g., Entertainment, Cloud, Utilities) and a subscription plan (e.g., Netflix Basic, Netflix Premium) from pre-populated service and plan tables.
REQ-F-12	The add and edit subscription forms must validate that all required fields are filled before submission and show clear error messages for missing or invalid inputs.

4.3 Dashboard & Analytics

4.3.1 Description and Priority

The dashboard is the first screen users see after logging in. It displays summary statistics and upcoming renewal information. All data is computed from the user's subscription records in the database. Priority: High.

4.3.2 Stimulus / Response Sequences

- User logs in -> Dashboard loads -> Frontend sends authenticated GET requests to the backend -> Backend queries the database -> Summary stats and upcoming renewals are returned and displayed as cards and tables.
- User views the dashboard -> Sees total number of active subscriptions, total monthly cost, upcoming renewals within 7 days, and financial exposure for the next 30 days.

4.3.3 Functional Requirements

Req. ID	Requirement
REQ-F-13	The dashboard must display the total number of active subscriptions for the logged-in user.
REQ-F-14	The dashboard must display the total monthly cost (SUM of cost for all active subscriptions) for the logged-in user.
REQ-F-15	The dashboard must display all subscriptions with a renewal date within the next 7 days as an 'Upcoming Renewals' list.
REQ-F-16	The dashboard must calculate and display financial exposure: the total expected spending from subscriptions renewing in the next 30 days.
REQ-F-17	Dashboard data must be fetched fresh from the backend every time the page is loaded. Stale cached data must not be shown.

4.4 Renewal Alert & Notification System

4.4.1 Description and Priority

This is one of the most important features from a backend engineering perspective. A node-cron background job runs daily and automatically generates alerts for subscriptions that are due to renew within 3 days. Notifications are stored and shown to users in the app. Priority: High.

4.4.2 Stimulus / Response Sequences

- Every day at midnight -> node-cron triggers the reminderJob.js -> Backend queries the database for all subscriptions where renewal_date - CURRENT_DATE <= 3 -> For each qualifying subscription, an ALERT record is inserted into the ALERT table.
- A NOTIFICATION record is then created linking the alert to the user, with a message and delivery channel.
- User logs in or visits the Alerts page -> Frontend fetches pending alerts from the backend -> Alerts are displayed sorted by date with the subscription name and renewal date.
- User views a notification -> Notification status is updated to 'read'.

4.4.3 Functional Requirements

Req. ID	Requirement
REQ-F-18	The backend must include a node-cron job that runs automatically every day at midnight (cron expression: 0 0 * * *).
REQ-F-19	On each run, the cron job must query the database for all active subscriptions where the renewal date is within the next 3 days and no alert has already been sent for that renewal period.
REQ-F-20	For each qualifying subscription, the cron job must insert a record into the ALERT table with alert_type set to 'renewal_reminder', the alert_date, and status 'pending'.
REQ-F-21	For each alert created, a corresponding NOTIFICATION record must be inserted into the NOTIFICATION table with a human-readable message and the delivery channel (e.g., 'in-app').
REQ-F-22	The Alerts page must display all pending alerts for the logged-in user, showing the subscription name, service, renewal date, and alert type.
REQ-F-23	When a user views their alerts, the notification status must be updated to 'read' in the database.

4.5 Payment Method Management

4.5.1 Description and Priority

Users can store multiple payment methods (credit/debit cards, UPI, etc.) and link them to their subscriptions. This helps track which payment method is being charged for each subscription. Priority: Medium.

4.5.2 Stimulus / Response Sequences

- User navigates to payment methods settings -> Adds a new payment method by specifying type, provider, last four digits, and expiry date -> Saved to database linked to their user account.
- When adding a subscription -> User selects from their saved payment methods via a dropdown.

4.5.3 Functional Requirements

Req. ID	Requirement
REQ-F-24	Users must be able to add multiple payment methods to their account, with fields: method_type (Credit Card, Debit Card, UPI, Net Banking), provider, last_four_digits, expiry_date, and is_default flag.
REQ-F-25	Users must be able to mark one payment method as their default. The default payment method should be pre-selected when adding a new subscription.
REQ-F-26	Users must be able to delete a payment method, provided it is not currently linked to any active subscription.
REQ-F-27	Each subscription must be linked to one of the user's saved payment methods at the time of creation.

4.6 Usage Log & Unused Subscription Detection

4.6.1 Description and Priority

The app allows users to log usage events for each subscription (e.g., 'watched a movie on Netflix', 'ran code on AWS'). Subscriptions with no recent usage can be flagged as potentially unused. Priority: Medium.

4.6.2 Stimulus / Response Sequences

- User logs a usage event for a subscription -> Backend stores the event in the USAGE_LOG table with date, type, and value.
- User views their subscription list -> Any subscription with no usage logged in the past 30 days is shown with a 'Possibly Unused' indicator.

4.6.3 Functional Requirements

Req. ID	Requirement
REQ-F-28	The app must allow users to log usage events for any of their subscriptions, including a usage_type and usage_value field.
REQ-F-29	The backend must be able to query the USAGE_LOG table to identify subscriptions that have had no usage events in the past 30 days.
REQ-F-30	Subscriptions identified as potentially unused must be visually highlighted in the subscription list with a warning indicator.

5. Other Non-Functional Requirements

5.1 Performance Requirements

Req. ID	Requirement
REQ-NF-01	The dashboard page must load and display all subscription statistics within 3 seconds on a connection with at least 5 Mbps speed.
REQ-NF-02	All REST API responses for list operations (e.g., GET /api/subscriptions) must respond within 500 milliseconds under normal database load.
REQ-NF-03	The node-cron job must complete its daily renewal check and alert creation within 30 seconds, even with up to 10,000 active subscriptions in the database.
REQ-NF-04	The database must use indexed queries for user_id on the subscriptions table to ensure query performance does not degrade as the number of records grows.
REQ-NF-05	The React frontend must render the dashboard UI within 2 seconds of receiving data from the API.

5.2 Safety Requirements

Since this is an academic project and not a certified financial management tool, outputs should not be relied on for financial or billing decisions:

- All financial summary figures on the dashboard must include a small disclaimer: 'Figures are estimates for informational purposes only.'
- REQ-NF-06: A disclaimer must be shown on the Dashboard and Financial Exposure section: 'Results are for academic/demo purposes only. Do not use as a replacement for official billing records.'

5.3 Security Requirements

Req. ID	Requirement
REQ-NF-07	All user passwords must be hashed using bcrypt before storage. The application must never store, log, or transmit plain text passwords.
REQ-NF-08	The JWT secret key must be stored in a .env environment variable file and must never be uploaded to the public GitHub repository. The .env file must be listed in .gitignore.
REQ-NF-09	All API calls between the frontend and backend in the production environment must use HTTPS only.
REQ-NF-10	All SQL queries involving user input must use parameterized queries to prevent SQL injection attacks.
REQ-NF-11	Any user-entered text displayed in the React UI must be properly escaped to prevent XSS (cross-site scripting) attacks.
REQ-NF-12	The backend must implement CORS restrictions so that only the deployed frontend URL is permitted to make API requests.
REQ-NF-13	The application must not expose sensitive user data (like password_hash, full card numbers, or internal IDs) in any API response to the frontend.

5.4 Software Quality Attributes

Attribute	What we aim for
Usability	The interface should be simple enough to use without any instructions. Forms must have clear labels, placeholder text, and inline validation error messages.
Reliability	If the database is temporarily unavailable, the backend should return a meaningful error response and not crash. The frontend must display a user-friendly error message rather than a blank screen.
Maintainability	The backend code is split into logical folders: controllers/, routes/, middleware/, models/, and scheduler/. Each module handles a single responsibility, making it easy to update or test independently.
Portability	The app must work on all major browsers (Chrome, Firefox, Edge, Safari) and on mobile phones. No browser-specific hacks should be used.
Testability	Business logic (e.g., financial exposure calculation, renewal date checking) should be written as pure functions in utility files so they can be unit tested without requiring a live database connection.
Availability	The app is hosted on Vercel (frontend) and Render (backend), both of which offer strong uptime records. We expect the app to be available during all demos and evaluation sessions.
Scalability	The database schema and backend API must support multiple concurrent users, each with thousands of subscriptions, without requiring major code changes.

5.5 Business Rules

- The project is released under the MIT License, meaning anyone can use or modify the code as long as they give credit.
- Each user's data is strictly private. A logged-in user can only access their own subscriptions, alerts, payment methods, and usage logs.
- The project landing page and GitHub repository mention Graphic Era (Deemed to be University) as the institution and list all three team members.
- If this project were ever used commercially, proper email/SMS notification APIs and a more robust billing tracker would need to be integrated.

6. Other Requirements

- Data: All application data is stored in a PostgreSQL relational database. The schema includes 10 entities: USER, CATEGORY, SERVICE, SUBSCRIPTION_PLAN, PAYMENT_METHOD, SUBSCRIPTION, PAYMENT, ALERT, NOTIFICATION, and USAGE_LOG. Foreign key constraints are enforced at the database level for all relationships.
- Language: Version 1.0 is in English only.
- License: The MIT License text and a copyright notice are included in the README on GitHub.
- Code Reuse: The financial exposure and renewal checking logic are written as standalone utility functions that can be reused or tested independently of the API routes.
- Deployment: The frontend can be deployed to Vercel by connecting the GitHub repository. The backend is deployed to Render by setting environment variables (DATABASE_URL, JWT_SECRET) in the Render dashboard. No manual server setup is required.
- Environment Variables: The following environment variables must be configured before running the application: DATABASE_URL (PostgreSQL connection string), JWT_SECRET (secret key for token signing), PORT (backend server port).

Appendix A: Glossary

Term / Acronym	What it means
SRS	Software Requirements Specification — a document that describes what a software system should do.
JWT	JSON Web Token — a compact, URL-safe token used to securely transmit user identity between the frontend and backend. Stored client-side and sent in every protected API request.
bcrypt	A password hashing library that applies a one-way cryptographic function with a salt to make stored passwords uncrackable even if the database is compromised.
node-cron	A Node.js library that schedules functions to run at specified times using cron expressions (e.g., 0 0 * * * means run at midnight every day).
REST API	Representational State Transfer API — a standard way of building HTTP endpoints that the frontend calls to read and write data.
CRUD	Create, Read, Update, Delete — the four basic operations performed on database records.
PostgreSQL	An open-source relational database system that stores all user and subscription data for this project.
React.js	A JavaScript library for building user interfaces using reusable components. Developed by Meta.
Express.js	A minimal web framework for Node.js that is used to define and handle the REST API routes in the backend.
Axios	A JavaScript library used to make HTTP requests from the React frontend to the Node.js backend.
Tailwind CSS	A CSS framework that lets you style HTML elements using utility classes directly in JSX.
SPA	Single-Page Application — a web app where one HTML page is loaded and content is updated dynamically as the user navigates, without full page reloads.
CORS	Cross-Origin Resource Sharing — a browser security rule that controls whether a web page can make requests to a different domain than the one it was loaded from.
Financial Exposure	In this project, the total expected subscription spending within the next 30 days, calculated by summing the cost of all subscriptions renewing in that window.
Renewal Reminder	An automated alert generated by the cron scheduler when a subscription's renewal date is 3 or fewer days away.
PII	Personally Identifiable Information — data that could identify a specific person, such as name, email, or phone number.
TBD	To Be Determined — used when a specific detail has not yet been decided.
HTTPS	HyperText Transfer Protocol Secure — the encrypted version of HTTP that protects data in transit between the browser and server.
Foreign Key	A database constraint that enforces a link between records in two tables, ensuring referential integrity.
CSE	Computer Science Engineering.

Appendix B: Analysis Models

B.1 How the Components are Organized

Smart Subscription Manager follows a 3-tier architecture. Here is how it all fits together:

Layer	Files / Components	What it does
Frontend Pages	Login.jsx, Register.jsx, Dashboard.jsx, AddSubscription.jsx, EditSubscription.jsx	Each page handles the UI for its specific feature and communicates with the backend via Axios API calls.
Frontend Components	Navbar.jsx, SubscriptionCard.jsx, DashboardStats.jsx	Reusable UI components used across multiple pages.
Frontend State	AuthContext.jsx, api.js	React Context API manages authentication state (user, token, login(), logout()). Axios service handles all API calls.
Backend Routes	authRoutes.js, subscriptionRoutes.js	Define the REST API endpoints and map them to the correct controller functions.
Backend Controllers	authController.js, subscriptionController.js	Business logic for each feature: validate input, query the database, return the response.
Middleware	authMiddleware.js	Verifies JWT on every protected route before the request reaches the controller.
Scheduler	reminderJob.js	node-cron job that runs daily, checks renewal dates, and inserts alert and notification records.
Database	db.js, PostgreSQL tables	10 relational tables storing all application data. Connections managed via a PostgreSQL connection pool.

B.2 How Data Flows Through the App

When a user registers: Browser sends POST /api/auth/register with name, email, password -> Backend hashes password with bcrypt -> User record inserted into USER table -> JWT generated and returned -> Frontend stores token in AuthContext -> User redirected to dashboard.

When a subscription is added: User fills form and submits -> Axios sends POST /api/subscriptions with JWT -> authMiddleware verifies token -> subscriptionController inserts record into SUBSCRIPTION table with all foreign key references -> Success response returned -> Frontend refreshes subscription list.

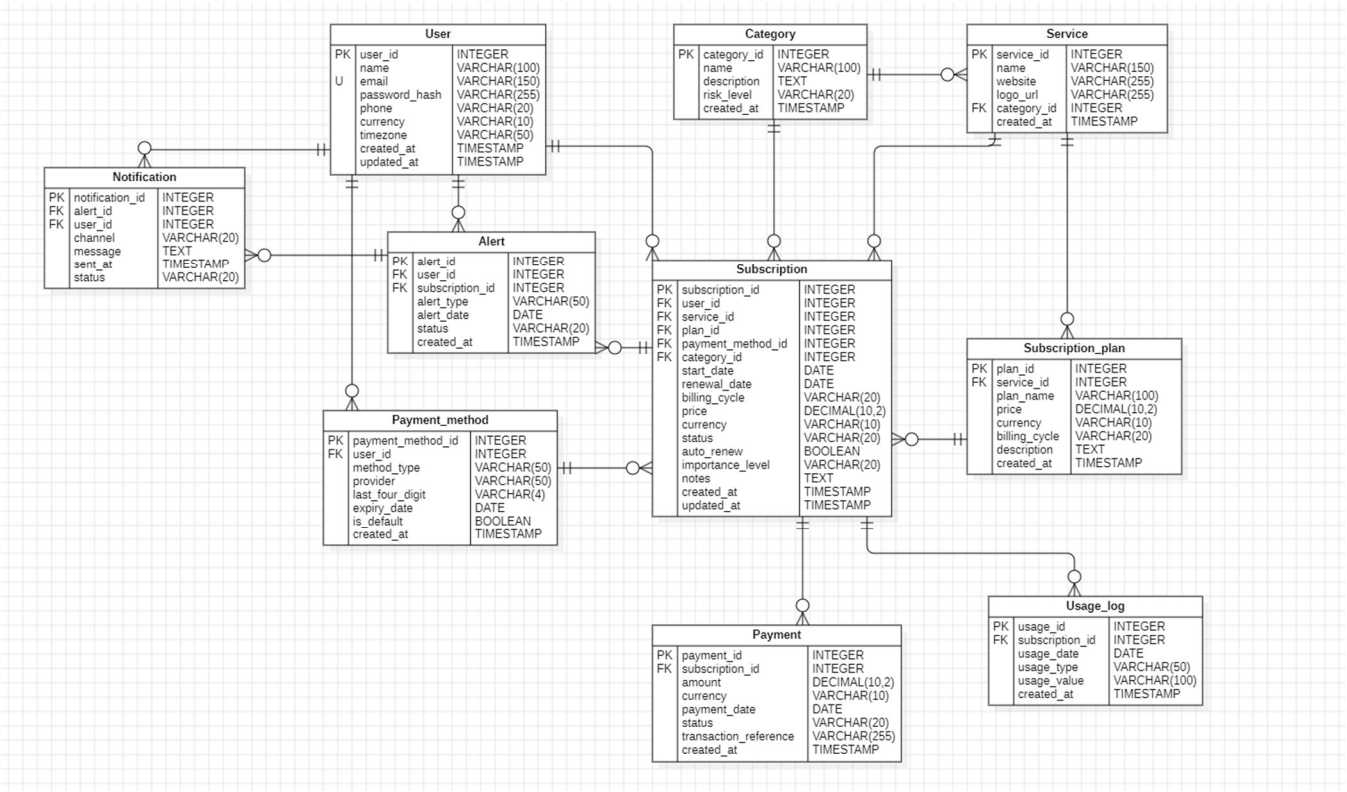
When the cron job runs: node-cron triggers reminderJob.js at midnight -> Backend queries database for subscriptions WHERE renewal_date - CURRENT_DATE <= 3 -> For each result, an ALERT record is inserted -> A NOTIFICATION record is linked to the alert and user -> Next day, user logs in and sees alerts on the Alerts page.

When the dashboard loads: Frontend sends authenticated GET requests to /api/dashboard -> Backend executes SQL queries: COUNT(*) for total subscriptions, SUM(cost) for monthly total, SELECT for renewals within 7 days, SUM(cost) for renewals within 30 days -> All results returned as JSON -> Dashboard components render the statistics.

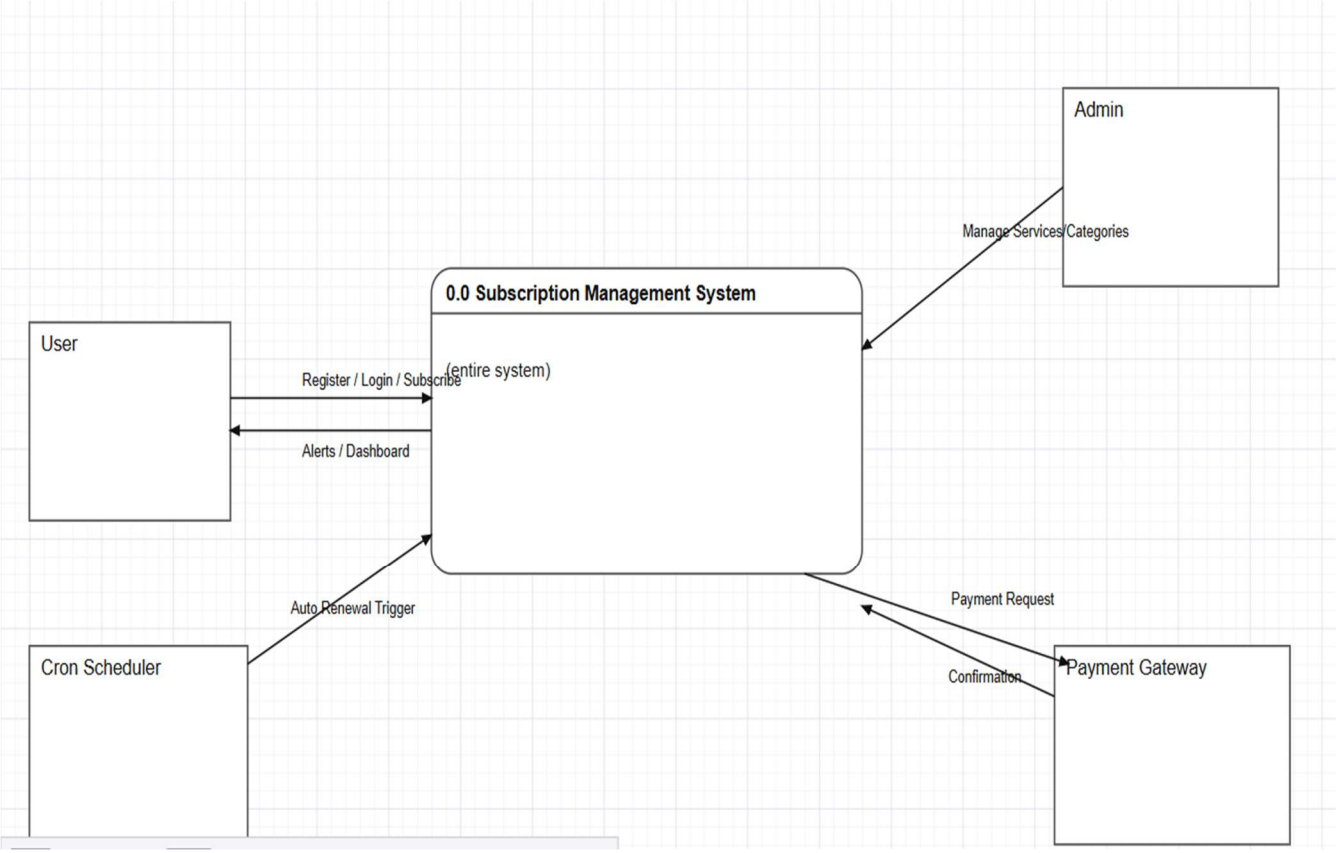
Appendix C: To Be Determined List

TBD ID	Section	What still needs to be decided
TBD-01	4.4	We have not finalized whether renewal alerts will also be sent via email (using a service like Nodemailer or SendGrid) in addition to in-app notifications. For now, only in-app notifications are implemented.
TBD-02	2.4	The final database hosting choice between Railway and Supabase (PostgreSQL) has not been finalized. Both support free-tier PostgreSQL and are compatible with the pg library.
TBD-03	4.6	The exact definition of 'unused subscription' (e.g., no usage in past 30 days vs. past 7 days) and the threshold for flagging it are still to be decided during implementation.
TBD-04	5.1	Performance under high concurrent user load has not been tested. We have not yet measured how many simultaneous users the Render free-tier backend can handle.
TBD-05	4.5	It has not been decided whether users can manage payment methods as a dedicated settings page, or only when adding/editing subscriptions. This depends on UI design decisions during development.

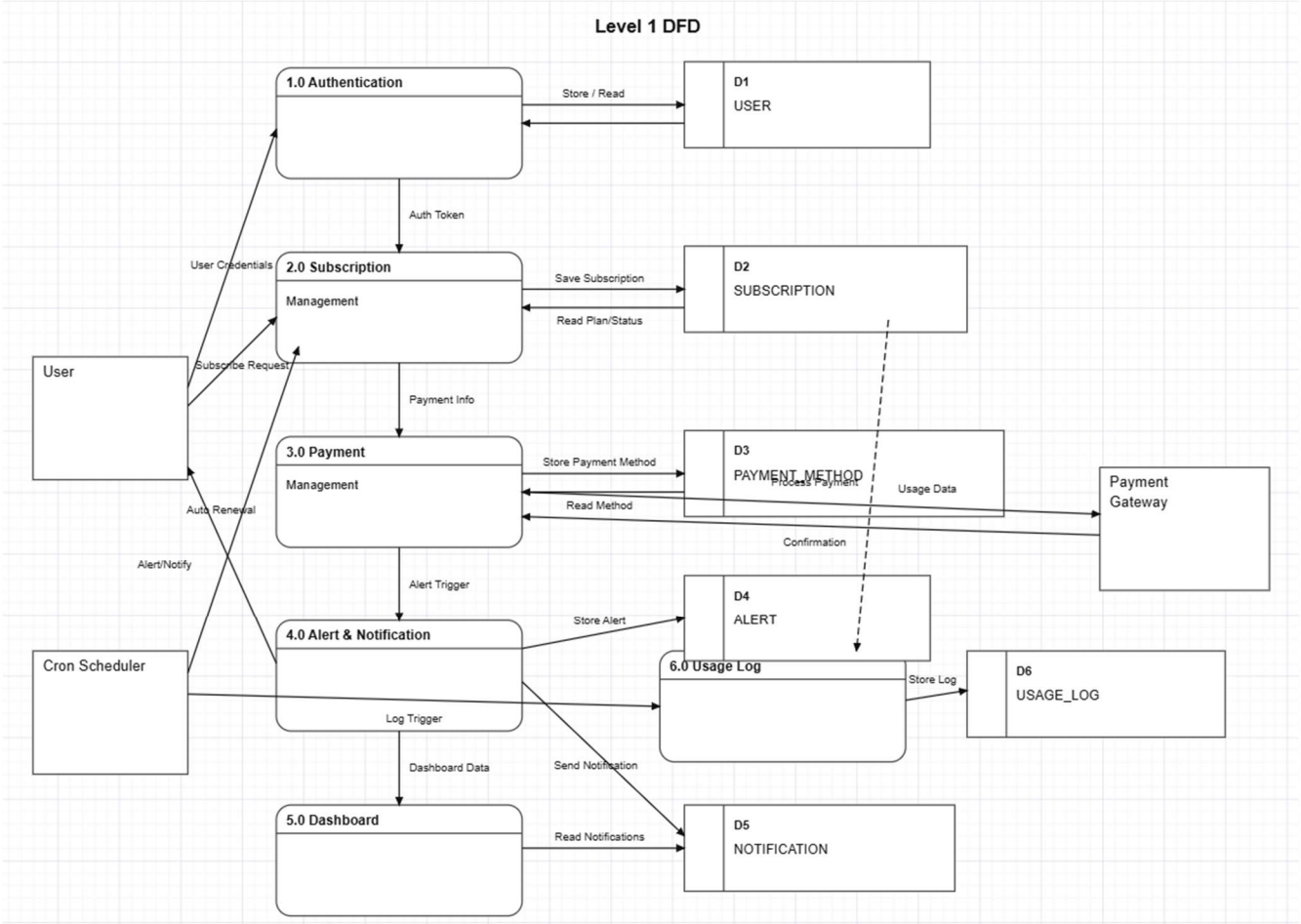
Appendix D: ERD AND DFD



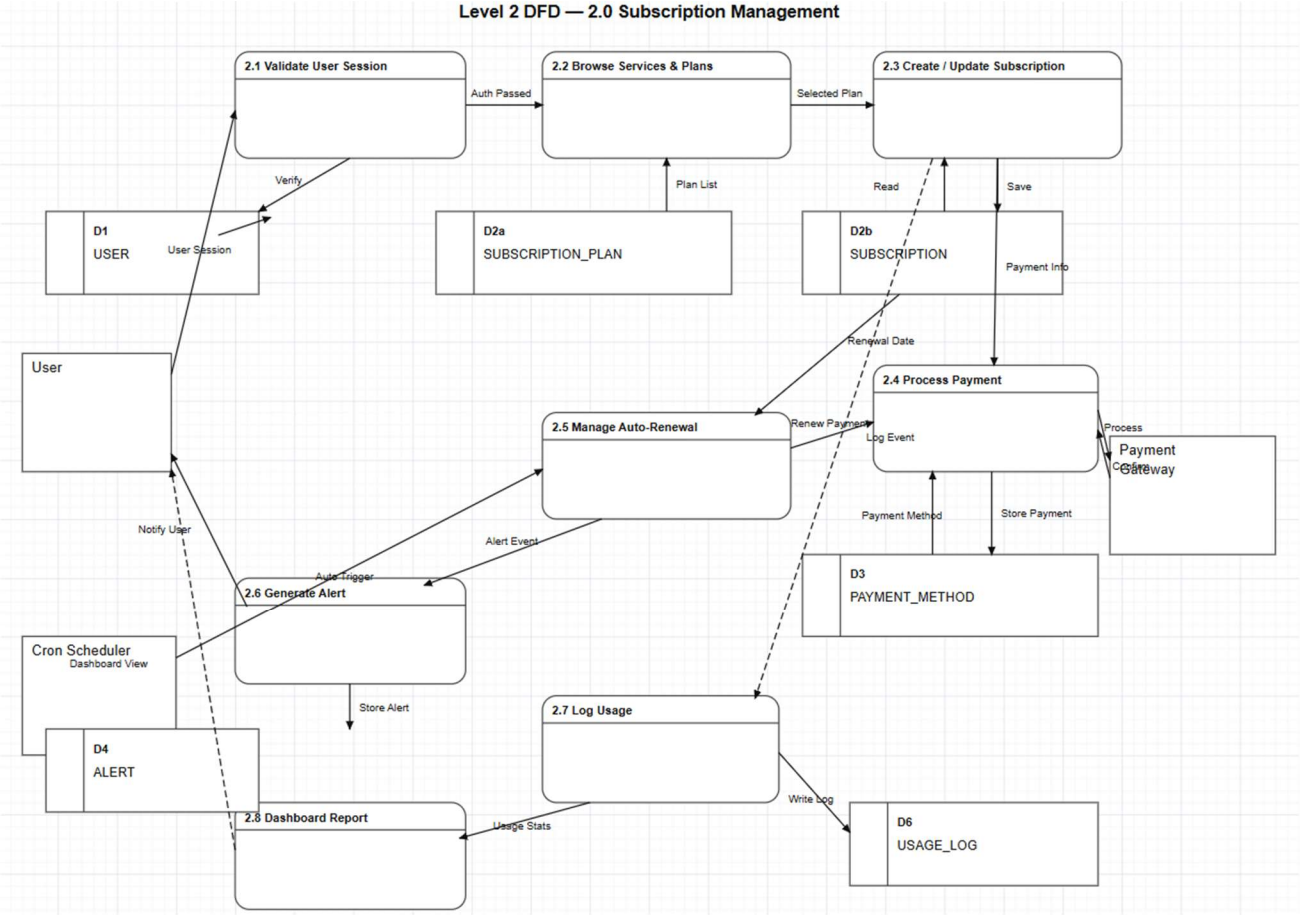
DFD – Level 0


Admin
Manage Services/Categories
0.0 Subscription Management System
(entire system)
User
Register / Login / Subscribe
Alerts / Dashboard
Auto Renewal Trigger
Cron Scheduler
Payment Request
Confirmation
Payment Gateway

DFD – Level 1


Level 1 DFD
1.0 Authentication
Store / Read
D1
USER
Auth Token
User Credentials
2.0 Subscription
Management
Save Subscription
Read Plan/Status
D2
SUBSCRIPTION
User
Subscribe Request
Payment Info
3.0 Payment
Management
Store Payment Method
D3
PAYMENT_METHOD
Process Payment
Usage Data
Payment
Gateway
Auto Renewal
Read Method
Confirmation
Alert/Notify
Alert Trigger
D4
ALERT
Store Alert
4.0 Alert & Notification
Cron Scheduler
6.0 Usage Log
Store Log
D6
USAGE_LOG
Log Trigger
Dashboard Data
Send Notification
5.0 Dashboard
Read Notifications
D5
NOTIFICATION

DFD – Level 2


Level 2 DFD — 2.0 Subscription Management
2.1 Validate User Session
Auth Passed
2.2 Browse Services & Plans
Selected Plan
2.3 Create / Update Subscription
Verify
Plan List
Read
Save
D1
USER
User Session
D2a
SUBSCRIPTION_PLAN
D2b
SUBSCRIPTION
Payment Info
Renewal Date
User
2.4 Process Payment
2.5 Manage Auto-Renewal
Renew Payment
Log Event
Process
Payment Gateway
Payment Method
Store Payment
Notify User
Alert Event
Auto-Trigger
2.6 Generate Alert
D3
PAYMENT_METHOD
Cron Scheduler
Dashboard View
Store Alert
2.7 Log Usage
D4
ALERT
Write Log
2.8 Dashboard Report
Usage Stats
D6
USAGE_LOG